Build Task — Project 4: Multi-Agent System — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — What you're building, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — What you're building, and the exact pieces

Think about the shape first, before any code: `supervisor → research_agent → analysis_agent → writer_agent → reviewer_agent`, where every specialist node hands control back to the supervisor when it's done, and the supervisor decides the next step from what's now in state. This is the same "one router, specialists report back" shape `supervisor_compare` already built — Project 4 just has 4 specialists instead of 2, and one of them (the reviewer) can send work backward, not just forward.

The key design decisions, matching the Build Task's own Requirements:

- **Shared vs. private state** — per Core Concepts' Shared state design section, state should hold each agent's *finished* output (`research_findings`, `analysis`, `draft`, `review_feedback`) and routing/status info (`revision_count`, `routing_log`), not any agent's raw internal reasoning.
- **The reviewer's decision** — it reads `draft` (and maybe `analysis`, to judge if the draft actually reflects it), and returns either `Command(goto=END)` with an "accepted" status, or `Command(goto="writer_agent")` with feedback, incrementing `revision_count` either way.
- **The hard limit** — checked in the reviewer node itself, before it even asks "is this good," the same defensive-check-first pattern `convergence_and_cost_cutting` practiced: if `revision_count >= MAX_REVISIONS`, stop and report "couldn't agree," full stop, regardless of what the reviewer would have said.

- **No redundant tool calls** — a Constraint from the README: each specialist's node should run exactly once per sub-task unless the reviewer specifically routes back to `writer_agent`; nothing should call `research_agent` twice for the same task.

Sketch `state.py`'s shape and all 5 node functions' signatures, then go one step further: think past "route to the next agent and check a counter" — ask **what does "a visible log of why the supervisor routed the way it did" (from the README's own Outputs section) actually require, beyond just a list of node names?** A bare `["research", "analysis", "writer", "reviewer"]` log tells you *what* ran, but nothing about *why* the supervisor thought that was the right next step — which is exactly the gap `ambiguous_routing` found matters once a routing decision isn't perfectly obvious.

The same question applies to the revision loop: the README's Test Cases table specifically requires a **clear** "couldn't agree" report when revision never settles, not just a stopped graph. That means carrying forward every rejection reason, not just the count — the exact pattern `convergence_and_cost_cutting` built, applied here for real.

Two extra design pieces answer both questions:

- **A routing log entry that includes the *reason*, not just the node name** — e.g. `{"step": "research_agent", "reason": "no research_findings in state yet"}` instead of just `"research_agent"` — so the final output can actually explain the supervisor's decisions, satisfying the Build Task's Outputs requirement directly.
- **Full LangGraph saved-state (a checkpointer), carried over from Project 3** — per the Build Task's own Requirements, wire in `from langgraph.checkpoint.memory import MemorySaver` (or a persistent equivalent) and compile with `checkpointer=memory`, passing a `thread_id` in config — this is what lets a run be paused, inspected, or resumed, the same saved-state discipline Doc10/Project 3 already established, now carried through a 4-agent graph instead of a single agent.

```
from langgraph.checkpoint.memory import MemorySaver

memory = MemorySaver()
graph = builder.compile(checkpointer=memory)

config = {"configurable": {"thread_id": "task-1"}}
result = graph.invoke(initial_state, config=config)
```

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

state.py

```
from typing import TypedDict

class SharedState(TypedDict):
    task: str
    research_findings: str
    analysis: str
    draft: str
    review_feedback: str
    accepted: bool
    revision_count: int
    routing_log: list
```

agents/supervisor.py

```
from langgraph.types import Command

MAX_REVISIONS = 3

def supervisor(state):
    log = state["routing_log"]
    if not state.get("research_findings"):
        return Command(
            update={"routing_log": log + ["research_agent"]},
            goto="research_agent",
        )
    if not state.get("analysis"):
        return Command(
            update={"routing_log": log + ["analysis_agent"]},
            goto="analysis_agent",
        )
    if not state.get("draft"):
        return Command(
            update={"routing_log": log + ["writer_agent"]},
            goto="writer_agent",
        )
    if state.get("accepted"):
        return Command(goto="__end__")
    if state["revision_count"] >= MAX_REVISIONS:
        return Command(goto="__end__")
    return Command(
        update={"routing_log": log + ["reviewer_agent"]},
        goto="reviewer_agent",
    )
```

`agents/reviewer_agent.py`

```
from langgraph.types import Command

MAX_REVISIONS = 3

def reviewer_agent(state):
    # your turn: judge state["draft"]; if it passes, set accepted=True and
    # go back to supervisor; if not, and revision_count < MAX_REVISIONS,
    # bump revision_count, save review_feedback, and route to writer_agent
    ...
```

Fill in `reviewer_agent`, `research_agent`, `analysis_agent`, and `writer_agent` yourself (each is small: do one job, update its field, `goto="supervisor"`), wire them into `graph.py` with `StateGraph(SharedState)`, and run a normal task through `main.py`, then go one step further, into the reasoned routing log below:

```
routing_log entries become dicts with a reason, not bare strings:
    {"step": "research_agent", "reason": "no research_findings in state yet"}

reviewer keeps every rejection's feedback, not just the last one:
    history = state["review_feedback_history"]
    state["review_feedback_history"] = history + [feedback]

when revisions run out:
    final report includes revision_count, MAX_REVISIONS, and the full
    review_feedback_history, so a reader can see exactly what the reviewer
    kept objecting to across every attempt

graph.py:
    compile with a MemorySaver checkpointer and a thread_id, so the run
    is resumable / inspectable the same way Project 3's agent was
```

Here's almost the whole thing for the supervisor's reasoned log — fill in the missing piece yourself:

```
def supervisor(state):
    log = state["routing_log"]
    if not state.get("research_findings"):
        reason = "no research_findings in state yet"
        entry = {"step": "research_agent", "reason": reason}
        return Command(
            update={"routing_log": log + [entry]},
            goto="research_agent",
        )
    if not state.get("analysis"):
```

```
        # your turn: same shape as above, but for analysis_agent
        ...
    # ... continue the same pattern for writer_agent and reviewer_agent ...
```

Fill in the rest of `supervisor`'s reasoned routing, `reviewer_agent`'s full feedback history, and the `MemorySaver` wiring in `graph.py`, then compare all of your finished depths against the Solution.

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

Story: `"__end__"` is a magic string a typo can break silently; the imported `END` can't be misspelled. And the README's "never agrees" test case deserves a real check, not an eyeball. **If not:** a misspelled end marker would quietly end nothing, and the revision limit would go untested until it mattered.

```
# agents/supervisor.py
from langgraph.graph import END
from langgraph.types import Command
from state import SharedState

MAX_REVISIONS = 3

def supervisor(state: SharedState) -> Command:
    log = state["routing_log"]
    if not state.get("research_findings"):
        return Command(
            update={"routing_log": log + ["research_agent"]},
            goto="research_agent",
        )
    if not state.get("analysis"):
        return Command(
            update={"routing_log": log + ["analysis_agent"]},
            goto="analysis_agent",
        )
    if not state.get("draft"):
        return Command(
            update={"routing_log": log + ["writer_agent"]},
            goto="writer_agent",
        )
    if state.get("accepted"):
        return Command(goto=END)
    if state["revision_count"] >= MAX_REVISIONS:
```

```
        return Command(goto=END)
    return Command(
        update={"routing_log": log + ["reviewer_agent"]},
        goto="reviewer_agent",
    )
```

This test file uses the same plain style as every exercise in this document: call the graph, `print()` what happened, and compare it against a `# expected` comment — the same pattern `convergence_and_cost_cutting`'s loop-limit test already used, not a test framework.

```
# test_project4.py -- proving the "reviewer rejects, writer improves" case
from agents.supervisor import MAX_REVISIONS
from graph import build_graph

graph = build_graph()

initial_state = {
    "task": "research the benefits of remote work and write a short brief",
    "research_findings": "",
    "analysis": "",
    "draft": "",
    "review_feedback": "",
    "accepted": False,
    "revision_count": 0,
    "routing_log": [],
}

def check_revision_loop_recovers() -> None:
    # deliberately weak first draft: writer_agent's prompt is swapped for one
    # that produces a too-short, low-effort draft on the first pass only
    result = graph.invoke(initial_state)
    print(f"routing_log length: {len(result['routing_log'])}")  # >= 4
    accepted = result["accepted"]
    hit_limit = result["revision_count"] == MAX_REVISIONS
    print(f"accepted or hit the limit: {accepted or hit_limit}")  # True

def check_never_agrees_hits_limit() -> None:
    # reviewer configured with an unsatisfiable bar (see
    # convergence_and_cost_cutting)
    result = graph.invoke(initial_state)
    print(f"accepted: {result['accepted']}")  # False
    print(f"revision_count: {result['revision_count']}")  # == MAX_REVISIONS

if __name__ == "__main__":
    check_revision_loop_recovers()
    check_never_agrees_hits_limit()
```

Back to the Build Task · Hint 1 · Hint 2 · Solution